

ECE391
Computer System Engineering
Lecture 14

University of Illinois at Urbana–Champaign

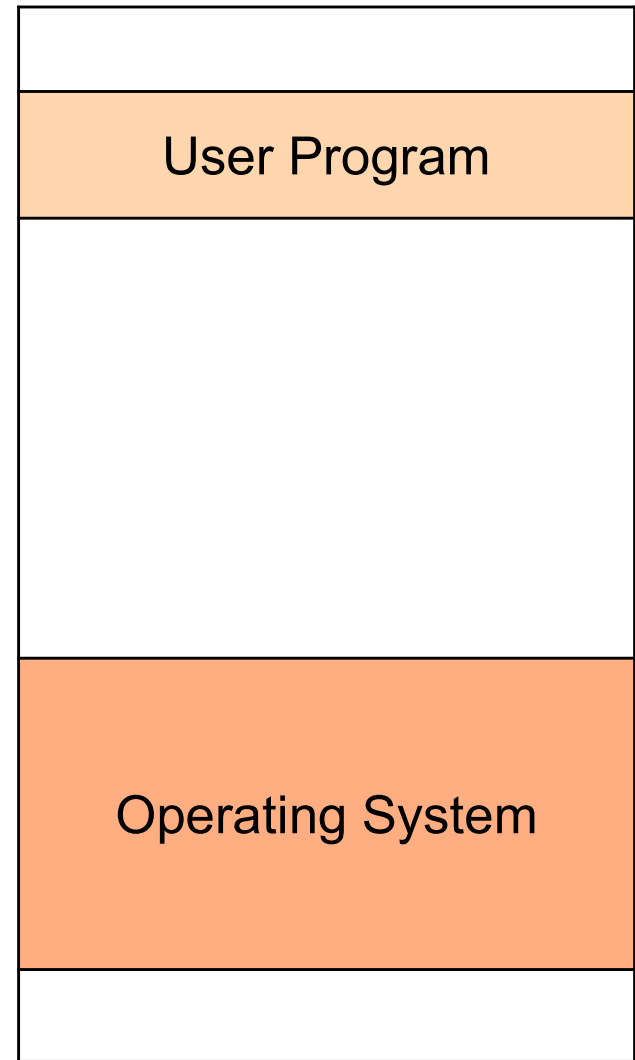
Spring 2024

Lecture Topics

- Virtual memory motivation
- x86 support for VM
 - protection model
 - segmentation
 - paging

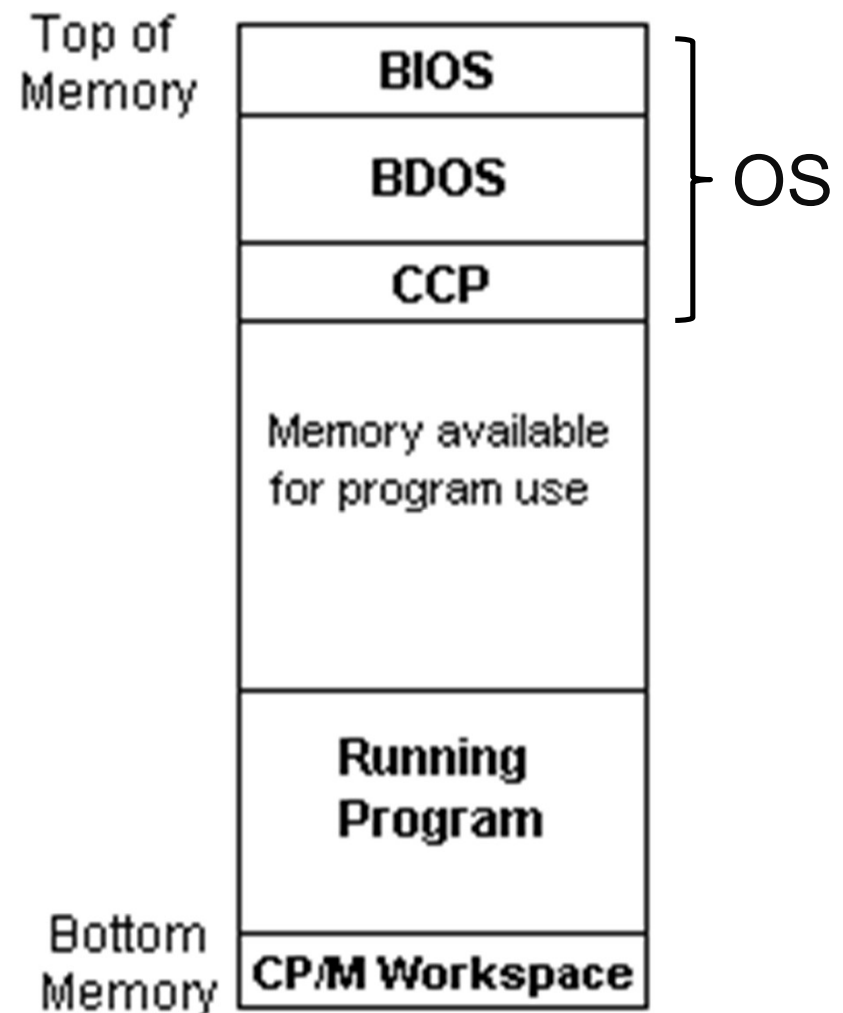
In the Beginning ...

- User program loaded into memory at designated address and executed
 - E.g. 0x3000 on LC-3
- Operating system acts essentially as a hardware abstraction layer



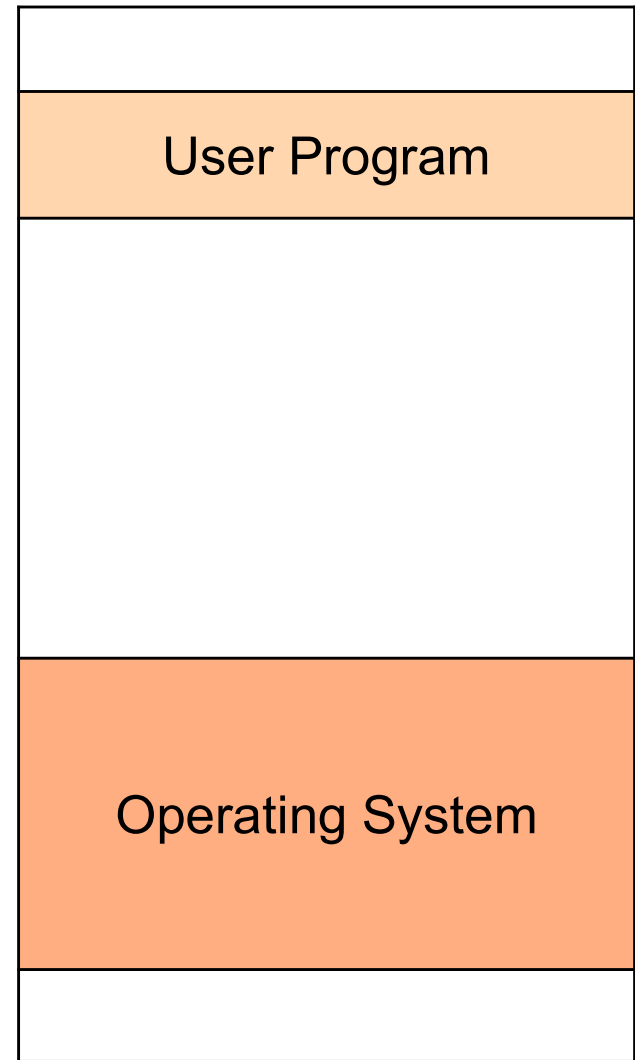
CP/M Memory Map

- CP/M operating system released in 1974 for Intel 8080/85 systems
 - Ported to 8086 and other CPUs
- User program loaded at 0x100
- OS in high memory



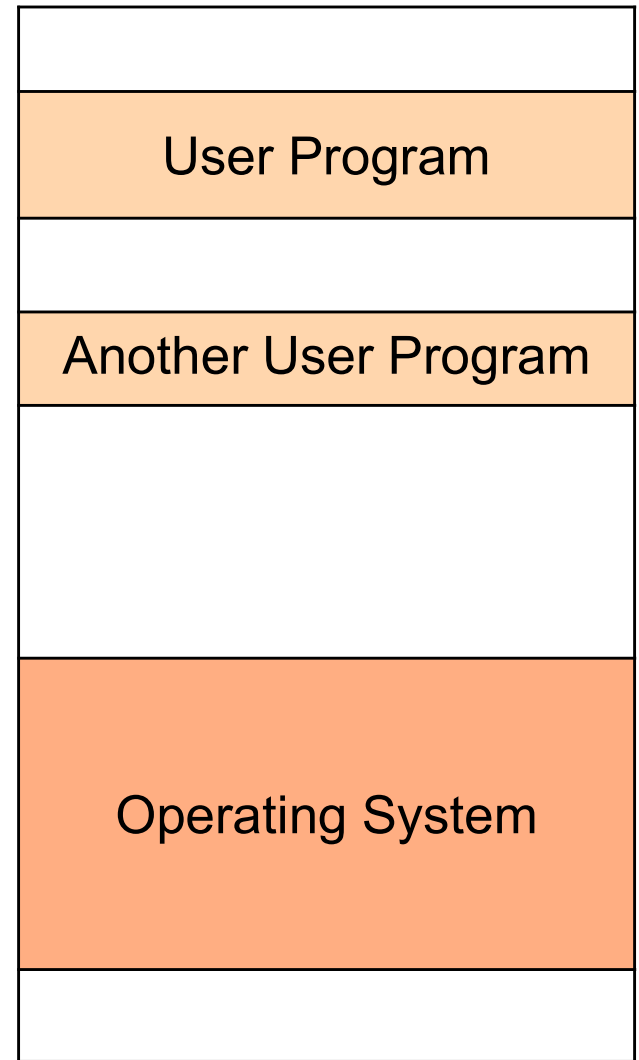
In the Beginning ...

- User program loaded into memory at designated address and executed
 - E.g. 0x3000 on LC-3
- Operating system acts as a library of drivers (with interrupt handlers)
- Want to run multiple programs at the same time



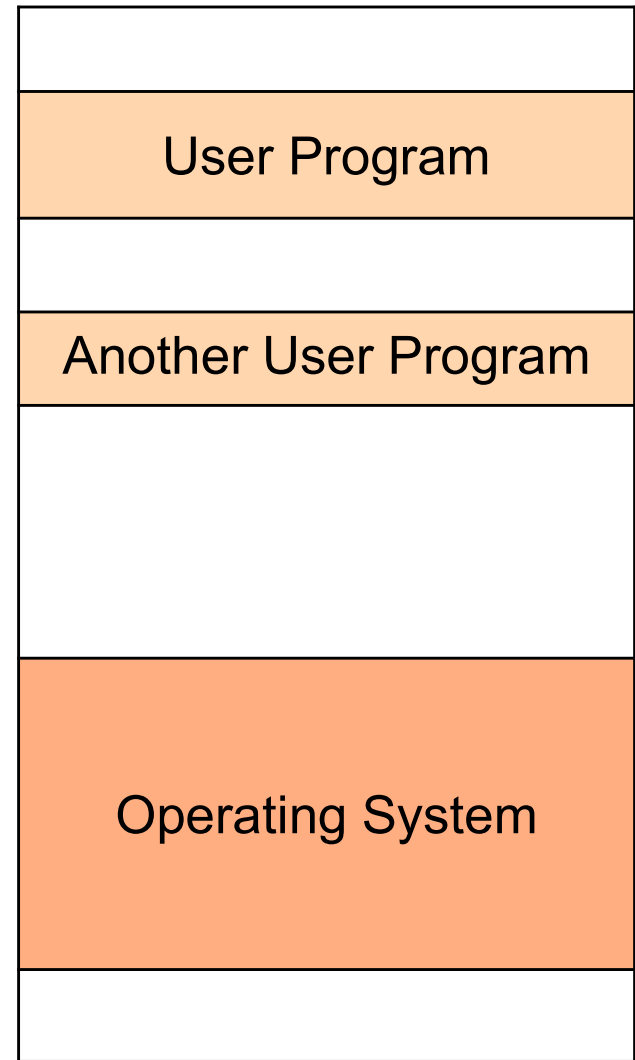
Multiprogramming

- **Multiprogramming:**



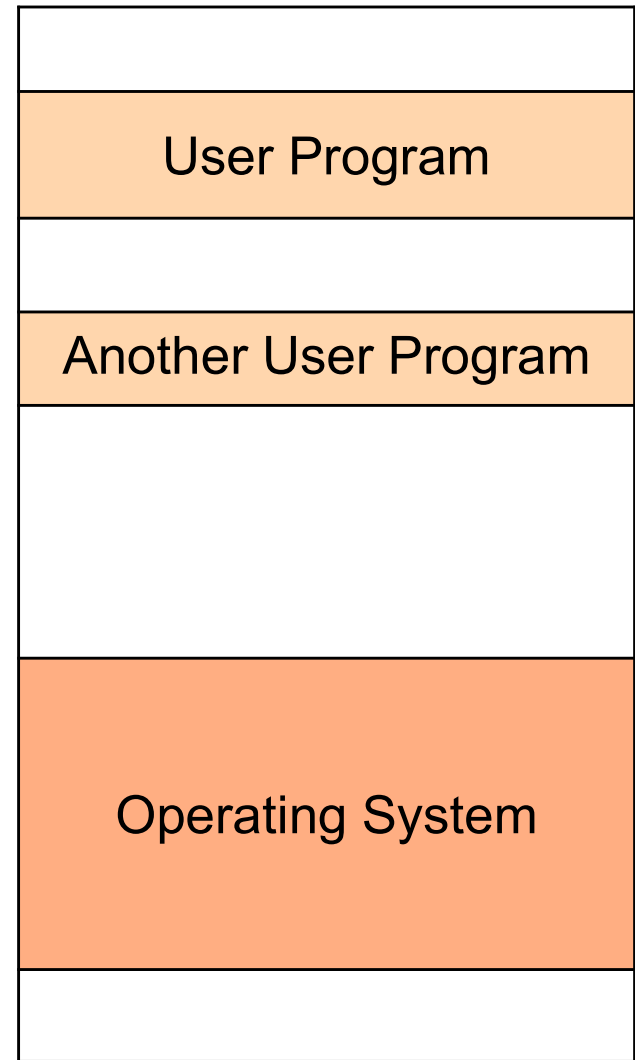
Multiprogramming

- **But:** Programs written with assumption they would be loaded as specific address
- *Solution 1:*
- *Solution 2:*



Multiprogramming

- **But:** A misbehaving program can corrupt another's memory (or OS)
- *Solution: ... ?*
- Need a way to limit what memory a program can access
- Possible to do in software only ...
 - Rewrite all memory reference instructions to check access
 - Called software *sandboxing*
- ... but a lot easier to do with some help from the hardware



Segmentation

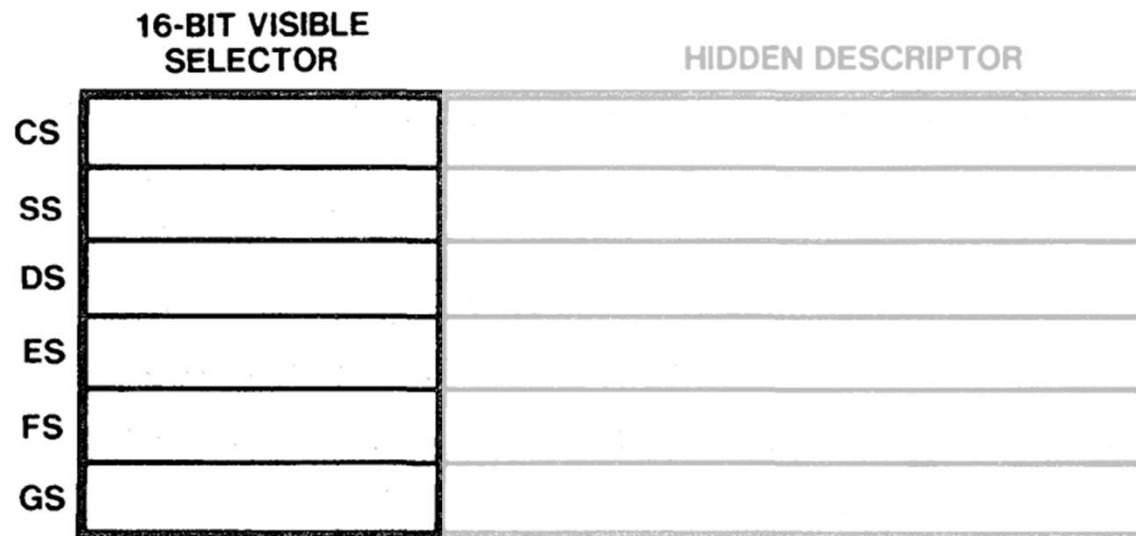
- Instead of a single flat memory space, independent pieces of memory, called *segments*
 - Each memory segment has a size
 - Each is it's own address space
- Memory references have two parts
 - Selector specifies which segment
 - Offset specifies offset within segment
 - Called **logical addresses** in Intel terminology
 - Analogy: disk organized as files (using file system)
 - Address of a byte is specified by file path and offset in file

Segmentation

- Segmentation is sounds exotic, but is very natural
- C program views memory as separate pieces
 - Code segment containing executable code
 - Static data (global variables)
 - Stack
 - Memory returned by malloc
- We don't assume any relationship between them or where they are in memory

Intel 386 Segmentation Model

- A logical address consists of a 16-bit segment selector and 32-bit offset in the selected segment
 - The addresses we've been using in the course so far are the offset portion of a logical address
- Program has access to 6 segments at any given time specified by the segment selector registers



Meet The Segment Selector Registers

- CS: Segment selector register
 - Default segment for jumps, branches, and subroutine calls
- SS: Segment selector register
 - Default segment for references using ESP and EBP
- DS: Segment selector register
 - Default segment for most other data accesses
- ES: Extra Segment selector register
 - With EDI used for string copy instructions (e.g. MOVS)
- FS & GS: additional selectors for explicit references

Specifying Segments

- Segment selector can be overridden from default
- `mov %fs:(%eax), %ebx`
segment given in %fs, offset given in %eax
- `mov 8(%edx), %ecx` $\Leftrightarrow$ `mov %ds:8(%edx), %ecx`
- `mov 8(%ebp), %ecx` $\Leftrightarrow$ `mov %ss:8(%ebp), %ecx`
- Can change any segment selector except CS:
 - `movw $0x10, %ax`
`movw %ax, %ds`
 - Cannot move immediate to segment selector register

Segment Addressing Exercise

- Move byte
at offset 0x1
in segment selected by
selector value 0x20
to offset 0x3
in segment selected by
selector value 0x40

Logical To Linear Addresses

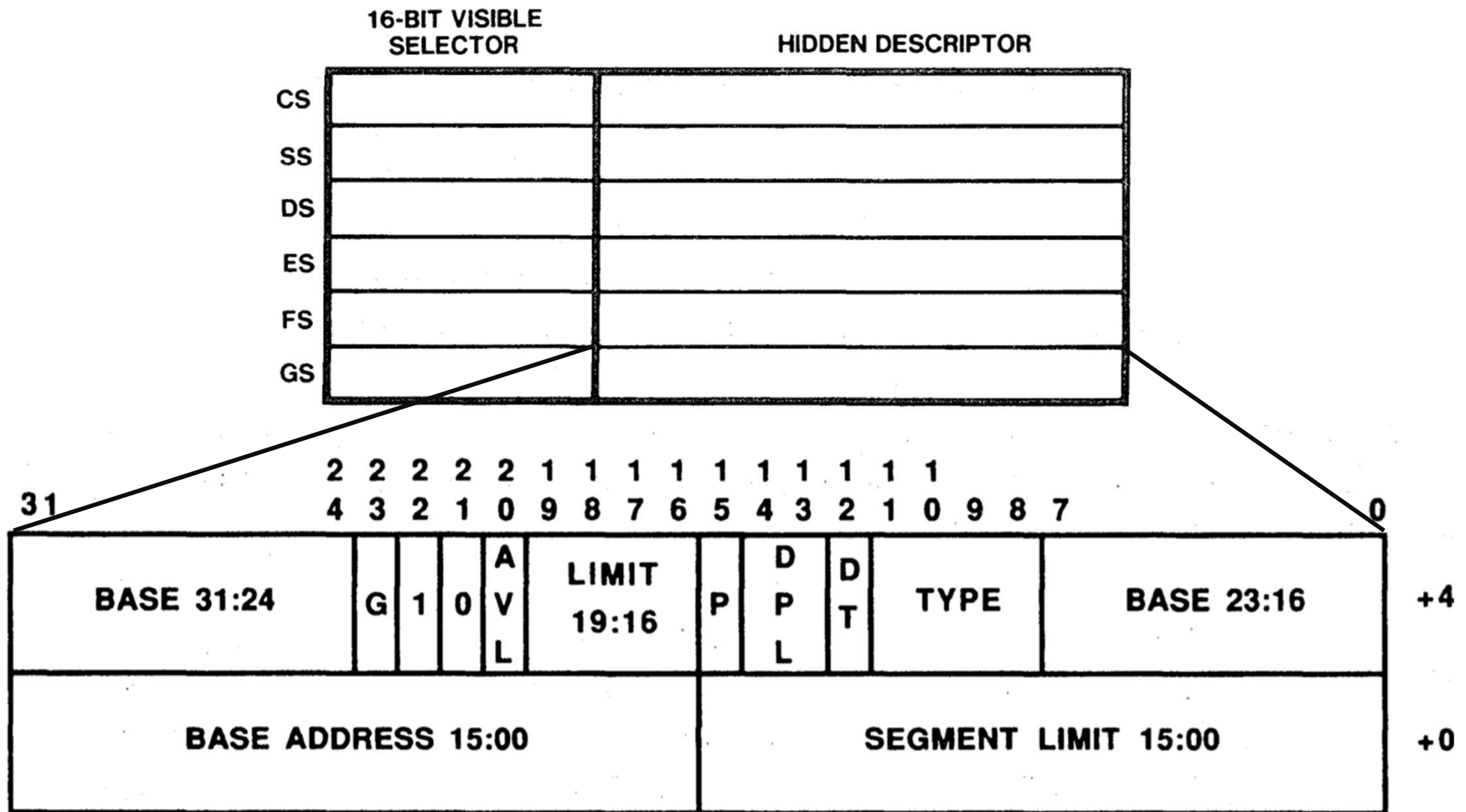
- Physical memory still uses physical (flat) addresses
 - Need to map logical addresses to physical addresses
- The x86 architecture has two levels of translation
- *This lecture:* segmentation (logical to linear)
- *Next lecture:* paging (linear to physical)
- Logical and linear are both a type of _____



Logical To Linear Addresses

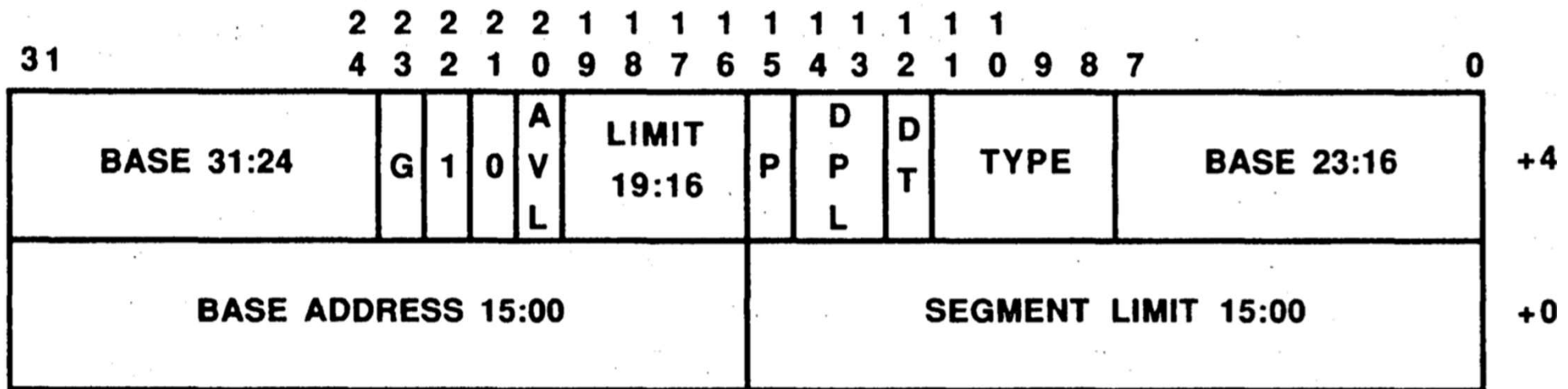
- Physical memory still uses physical (flat) addresses
 - Need to map logical addresses to physical addresses
- Each segment is mapped to a contiguous range of linear addresses
- Segment mapping is given by an 8-byte _____
- When we load segment selector register, CPU automatically fetches the corresponding descriptor and keeps it alongside for address translation

Logical To Linear Addresses



Segment Descriptors

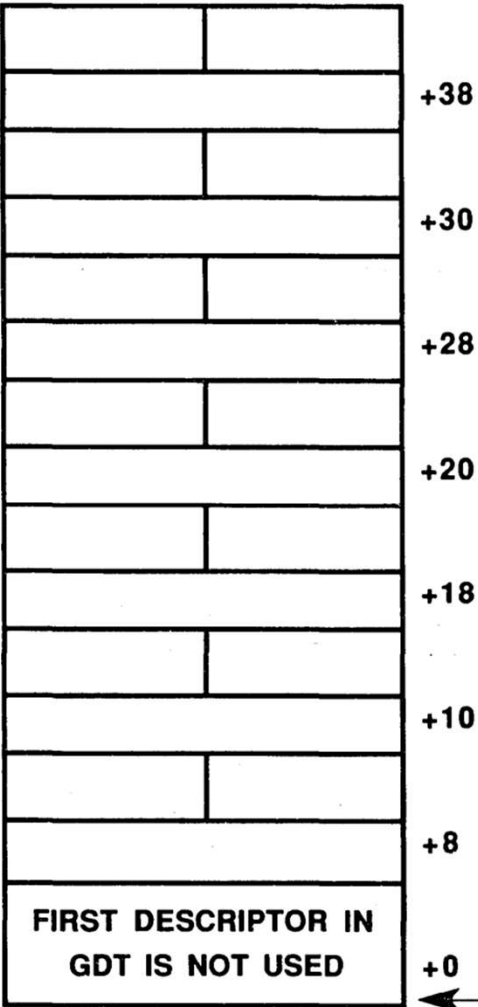
- Base: linear address of start of segment
- Limit: last valid offset in segment (size minus 1)
 - G = 0: limit in bytes, G = 1: limit in 4K increments
 - e.g. G=1, LIMIT=0 means valid offsets from 0 to 4095 inclusive
- DPL: Descriptor Privilege Level



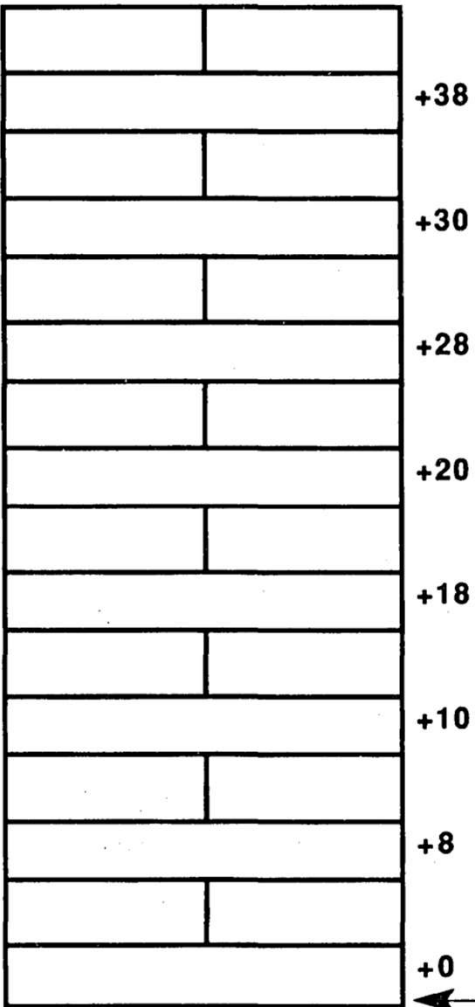
Descriptors Tables

- Segment descriptors fetched from descriptor table
- Global Descriptor Table: visible to all tasks
 - Linear address of table in GDTR, load using LGDT instr.
- Local Descriptor Table: per-task descriptor table
 - Selector of LDT in LDTR, load using LLDT instruction
 - LDT must be a segment in the GDT

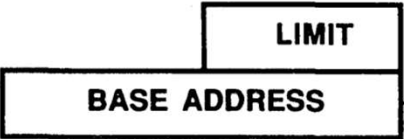
GLOBAL DESCRIPTOR TABLE



LOCAL DESCRIPTOR TABLE



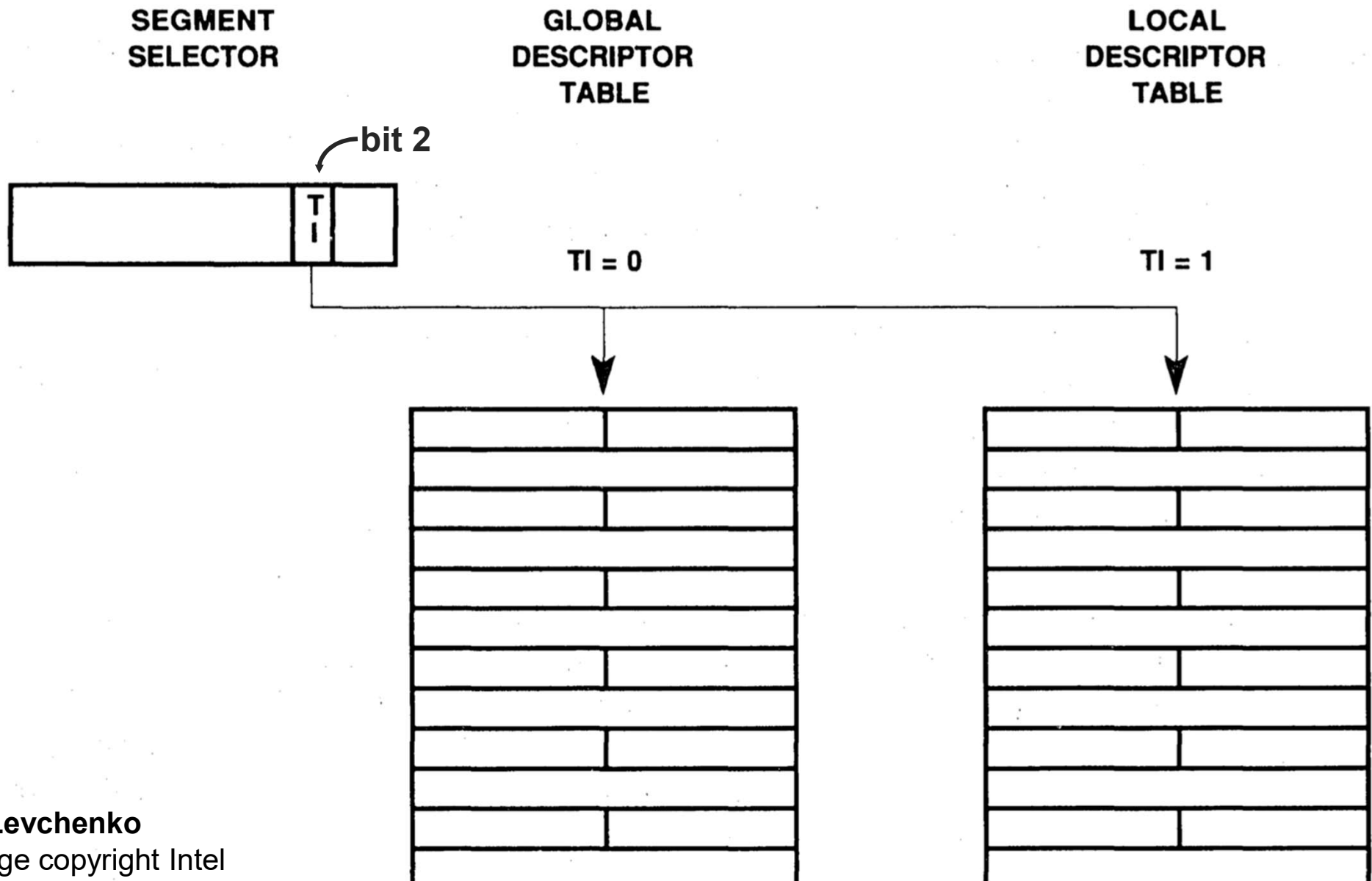
GDTR REGISTER



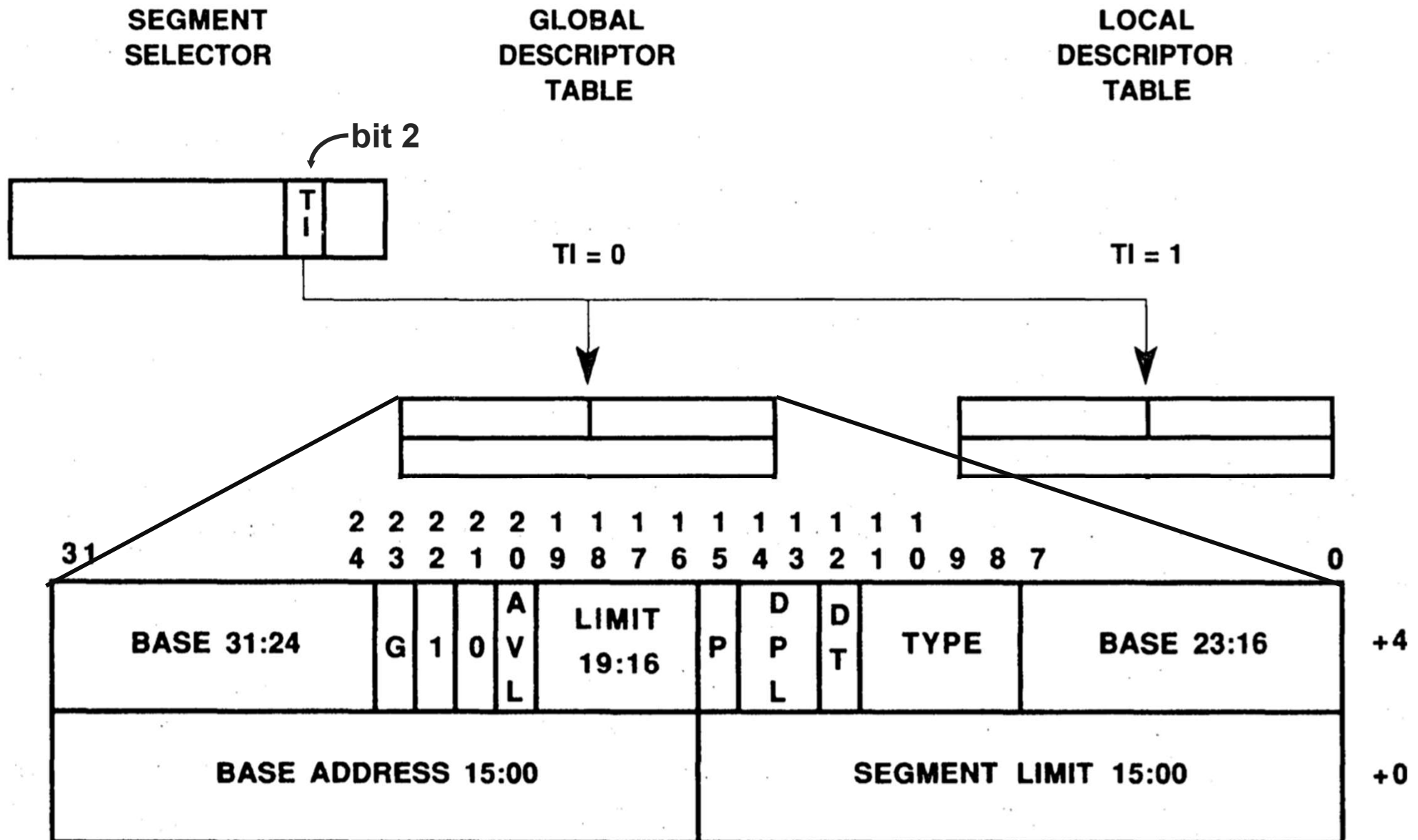
LDTR REGISTER



Descriptors Tables

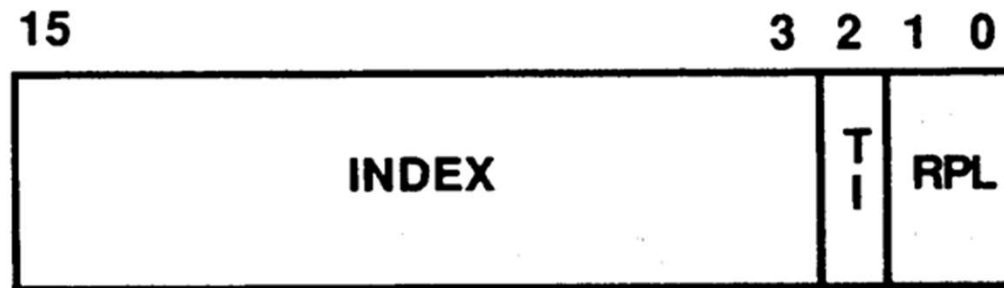


Descriptors Tables



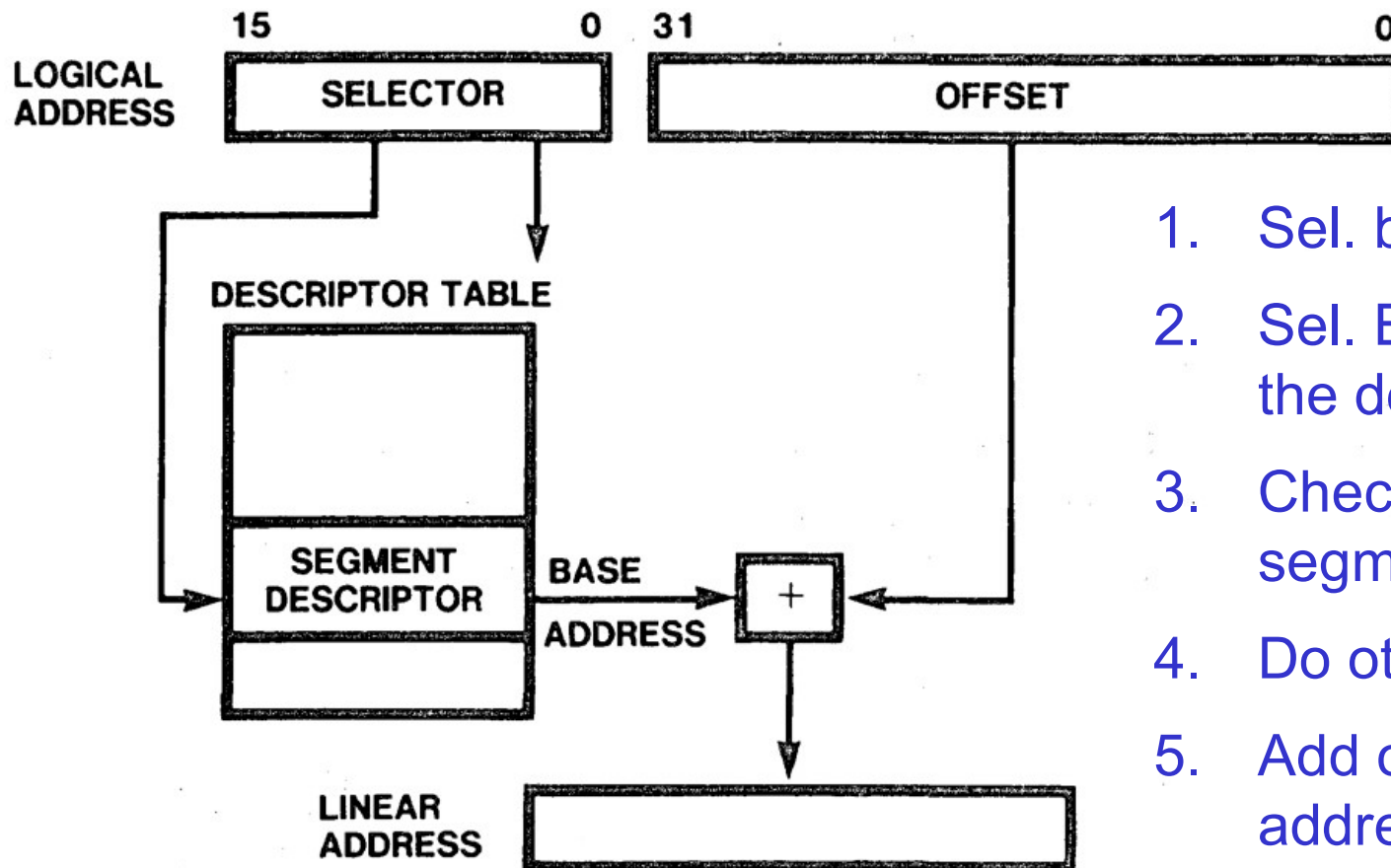
Segment Selectors

- Bits 1:0 of selector are requested privilege level (RPL)
 - Used for access control checks (will see in a few slides)
- Bit 2 is table indicator (TI)
 - TI=0 means GDT, TI=1 means LDT
- Bits 15:3 are index into descriptor table



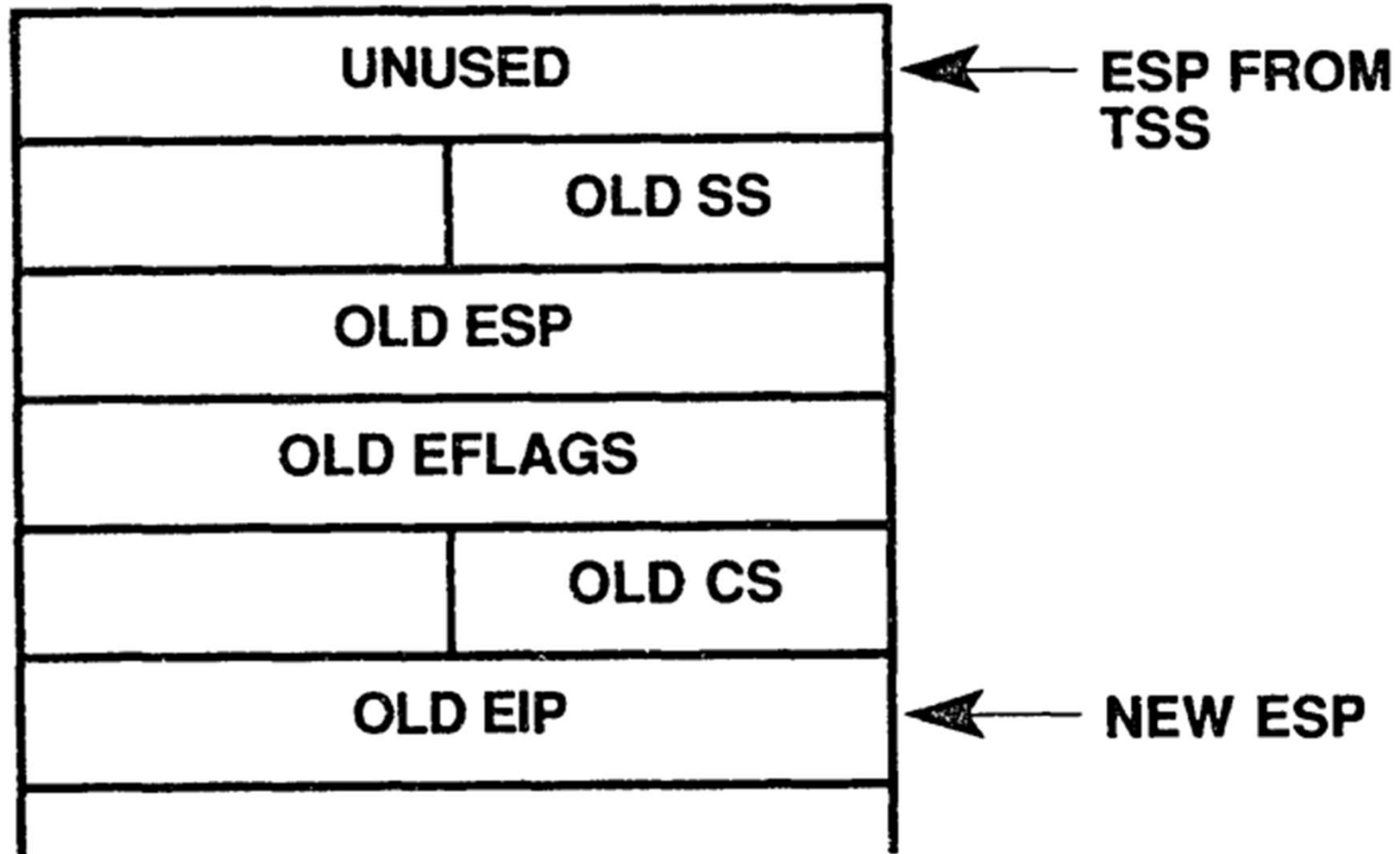
TI **TABLE INDICATOR (0 = GDT, 1 = LDT)**
RPL **REQUESTED PRIVILEGE LEVEL**
 (00 = MOST PRIVILEGED, 11 = LEAST)

Logical to Linear Address Translation

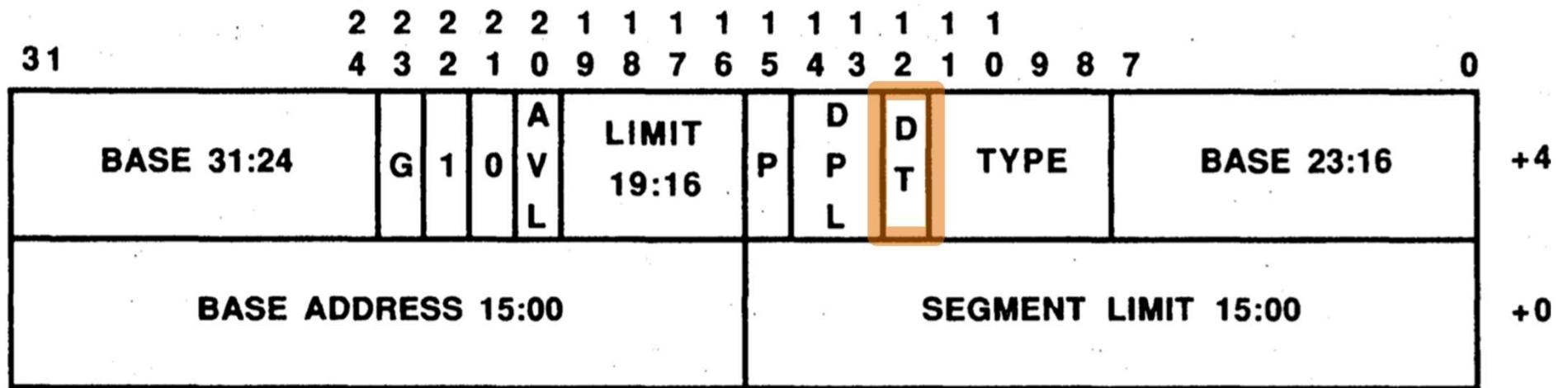


1. Sel. bit 2 indicates table
2. Sel. Bits 15:3 are index into the descriptor table
3. Check if offset is within segment limit
4. Do other protection checks
5. Add offset part of logical address to base address from descriptor

Stack on Entry to Syscall Handler

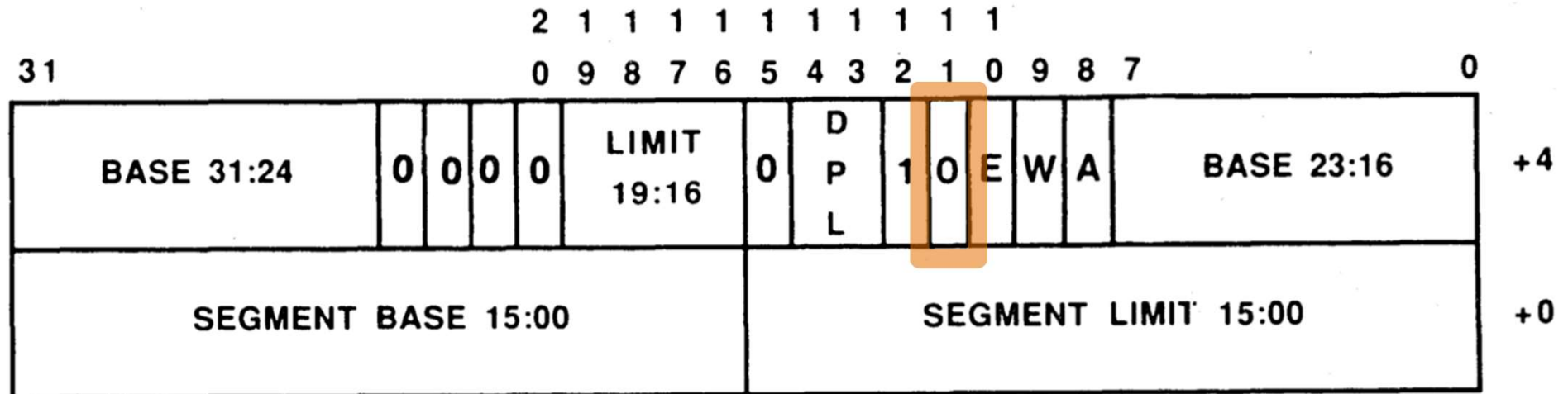


Segment Types



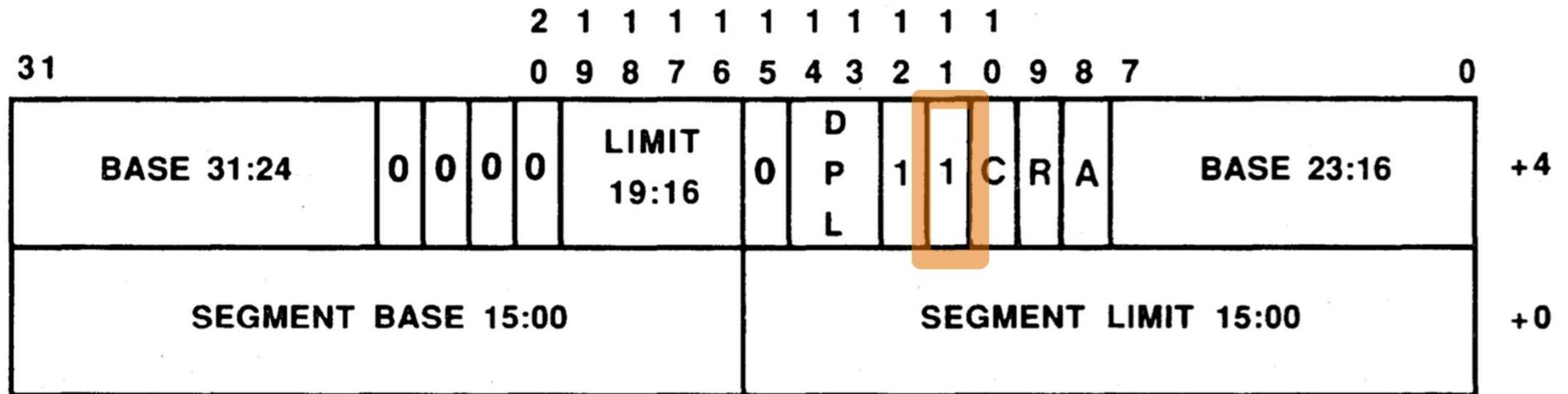
- DT=0: Special (“system”) segment
- DT=1: Not special (“application”) segment

Data Segment Descriptors



- E=1: expand-down; E=0: expand-up
 - Intended for re-sizeable stacks; see Intel docs for details
- W=1: writable; W=0: read-only
- A=1: accessed (set by CPU); A=0: not accessed

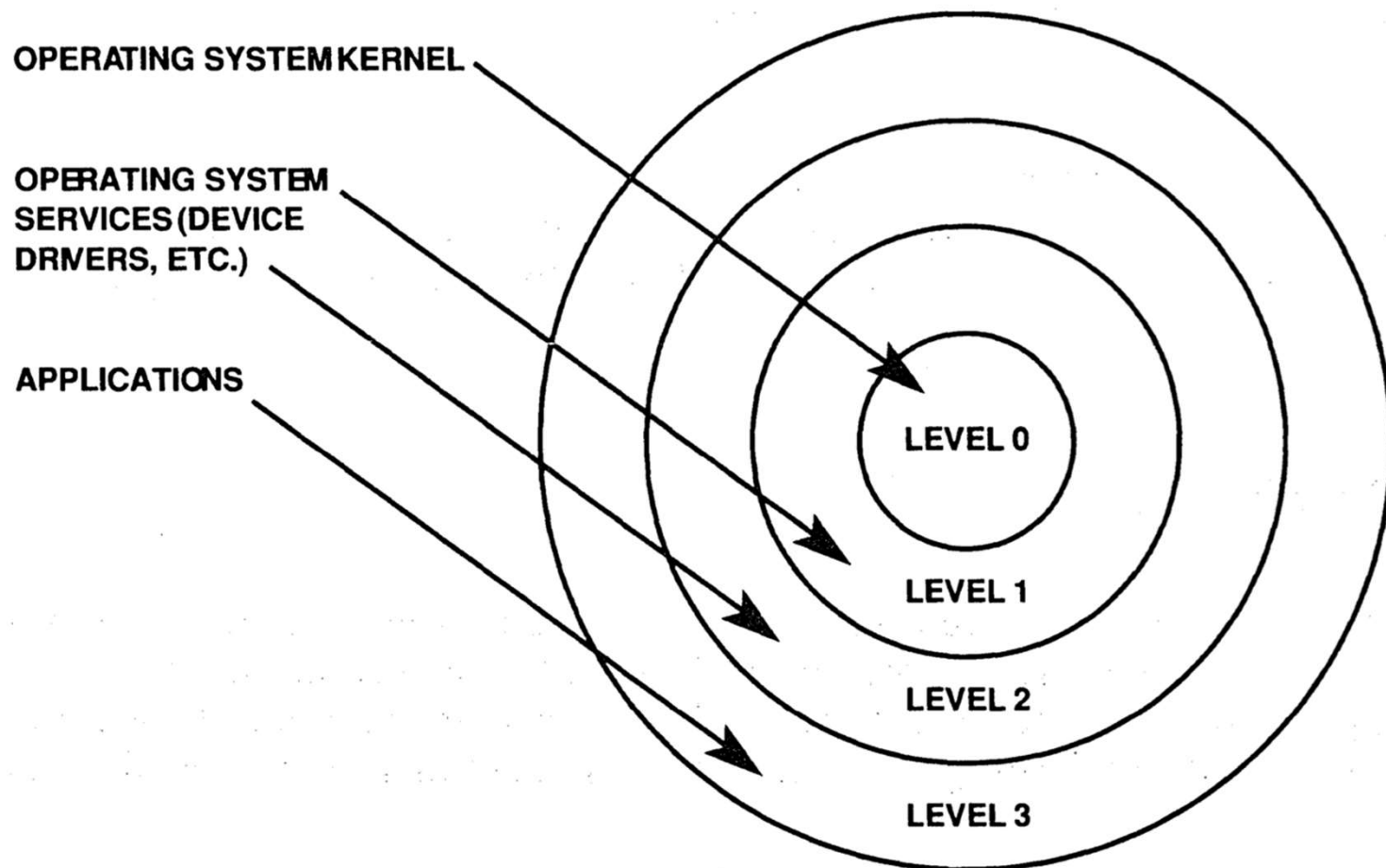
Code Segment Descriptors



- C=1: conforming; E=0: fixed privilege level
 - See Intel documentation for details
- R=1: readable; W=0: execute-only
- A=1: accessed (set by CPU); A=0: not accessed

Privilege Levels

- Recall that the IA-32 architecture has 4 privilege rings



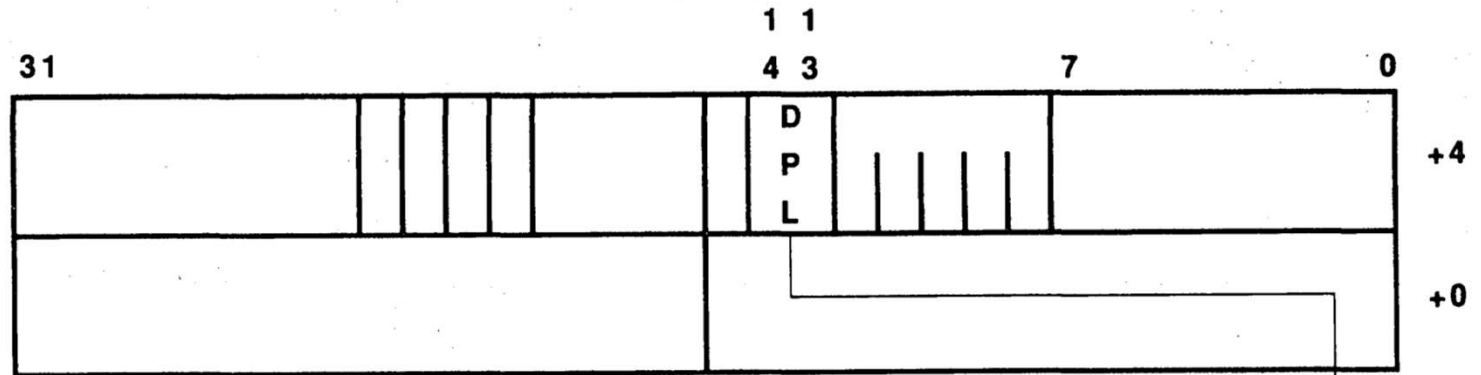
Memory Access Privilege Checking

- Consider simple memory read:

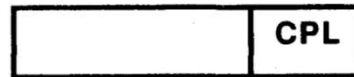
```
mov (%edx), %eax
```

- **RPL** (Requestor Privilege Level): bits 1:0 of selector of address program is trying to access
- **CPL** (Current Privilege Level): bits 1:0 of selector of currently executing code segment
 - Recall that user program cannot change CS directly
- **DPL** (Descriptor Privilege Level): bits 14:13 of descriptor of segment program is trying to access

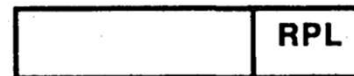
OPERAND SEGMENT DESCRIPTOR



CURRENT CODE SEGMENT REGISTER

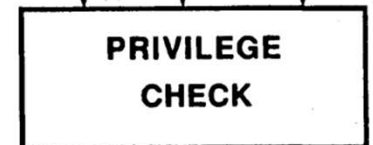


OPERAND SEGMENT SELECTOR



CPL
DPL
RPL

CURRENT PRIVILEGE LEVEL
DESCRIPTOR PRIVILEGE LEVEL
REQUESTED PRIVILEGE LEVEL



Memory Access Privilege Checking

- **RPL** (Requestor Privilege Level): bits 1:0 of selector of address program is trying to access
- **CPL** (Current Privilege Level): bits 1:0 of selector of currently executing code segment
- **DPL** (Descriptor Privilege Level): bits 14:13 of descriptor of segment program is trying to accessing
- Access allowed if

$$DPL \geq \max\{CPL, RPL\}$$

- *Does this make sense?*

$$DPL \geq \max\{CPL, RPL\}$$

Scenario	CPL	RPL	DPL	Allow?
OS tries to access its own stack				
OS tries to access user program stack				
User program tries to access its own stack				
User program tries to access OS stack				

- User program can make up any RPL it wants
 - User program can modify selector registers (except CS)
 - E.g. write a selector with RPL=0 to DS
- *So what is the point of RPL?*

The Point of RPL

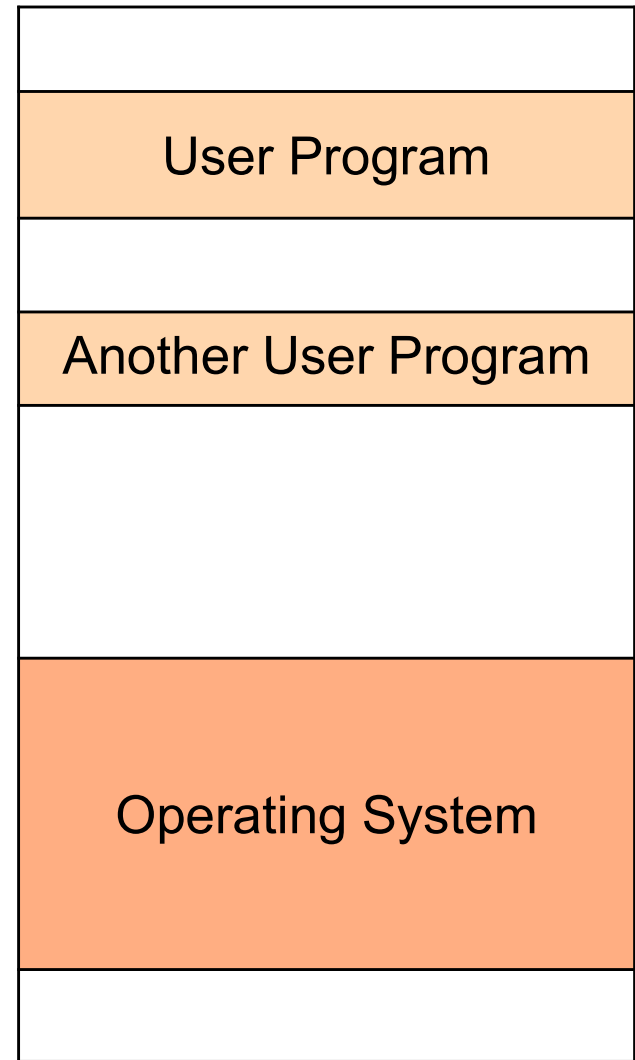
- Let's say user makes a `read()` system call and passes a (logical) pointer as the destination buffer ptr
 - User program can make up any RPL for the selector
- OS sets the selector RPL to 3 for user pointers it gets from system calls
- Prevents OS from accidentally using untrusted (user) pointer to reference privileged memory from OS code
 - Note that OS code is running with $CPL=0$

How Do We Use Segments?

- GDT and LDT managed by the operating system
 - LGDT and LLDT are privileged instructions
- OS keep track of physical memory and sets up segment descriptors for itself and programs

Simple GDT/LDT Configuration

- Code segment for OS code
- Data segment for OS static data, heap, and stack
- Code segment for user program code
- Data segments for user program static data, heap, and stack
- Logical addresses make programs independent of where they are placed in memory! *Why?*



Fine-Grained Segments

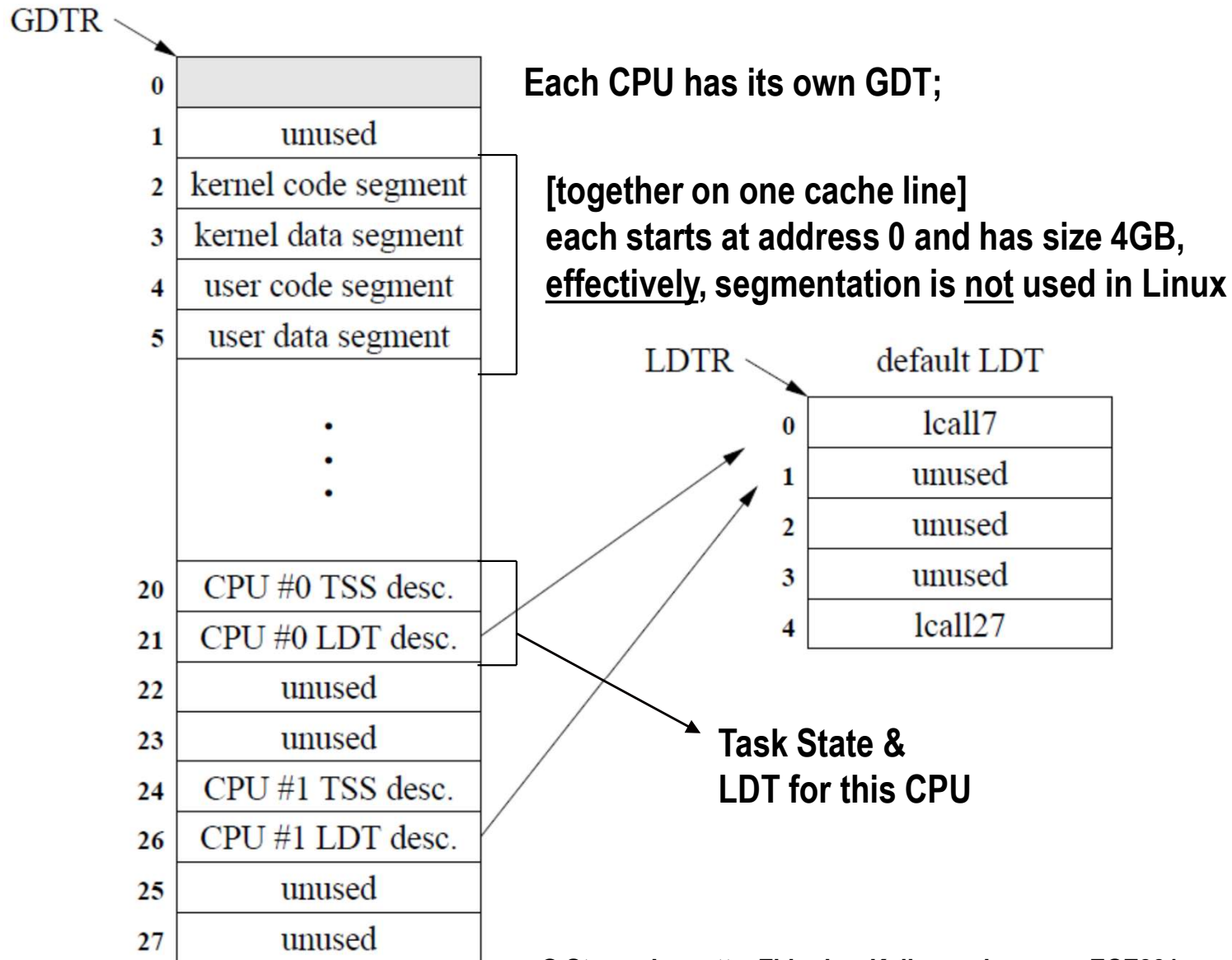
- Can we place the stack, heap, and static data in separate segments? (*Why would we want to do this?*)
- Yes, but programming becomes more complicated
- Need to pass around both parts of logical address
- Consider `strlen(const char * s)`
 - Need to pass both segment selector and offset to function
- Some compilers support this
 - Far pointers: full logical address
 - Near pointers: offset only
 - Assumes all objects of this type are in same segment
- Programmers do not like this

Segmentation

- There is little love for segmentation in this world
- GCC doesn't even support it
- Linux doesn't really use it
 - Sets up descriptors to cover all of memory
 - Segment base: 0x00000000
 - Segment limit: 0xFFFFF000 (G=1)
- Logical addresses behave like linear addresses

Linux Use of Segmentation

(details in `asm/segment.h`)



Where Does That Leave Us?

- Modern CPUs (except low-end microcontrollers) have a Memory Management Unit (MMU) that translates virtual to physical addresses
 - ARM, PowerPC, RISC-V, x86, etc.
- Segmentation is not used in modern CPUs
- Virtual addresses are linear, translated using paging

